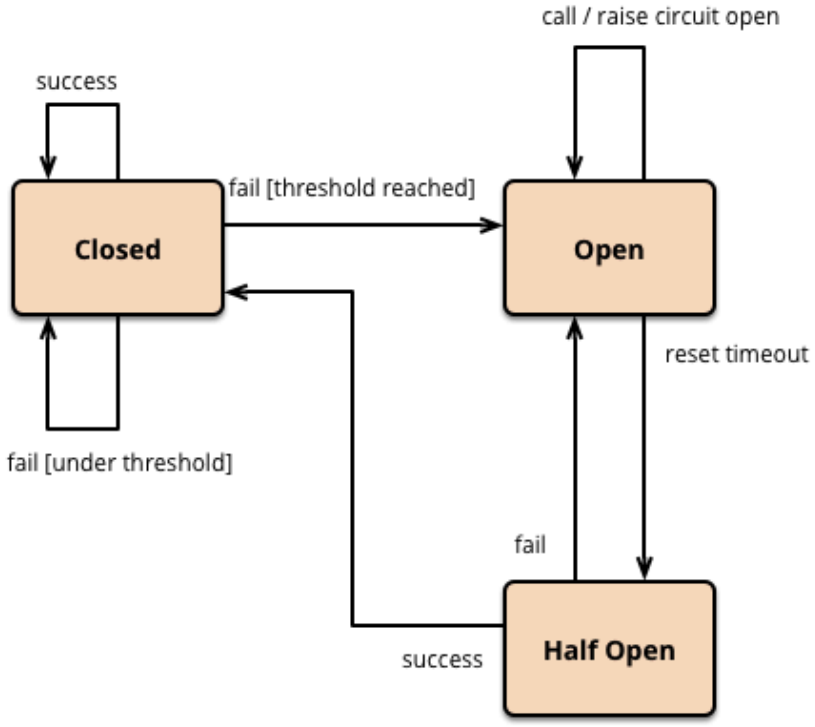
Circuit Breaker Pattern with Resilience4j and Spring Cloud Gateway

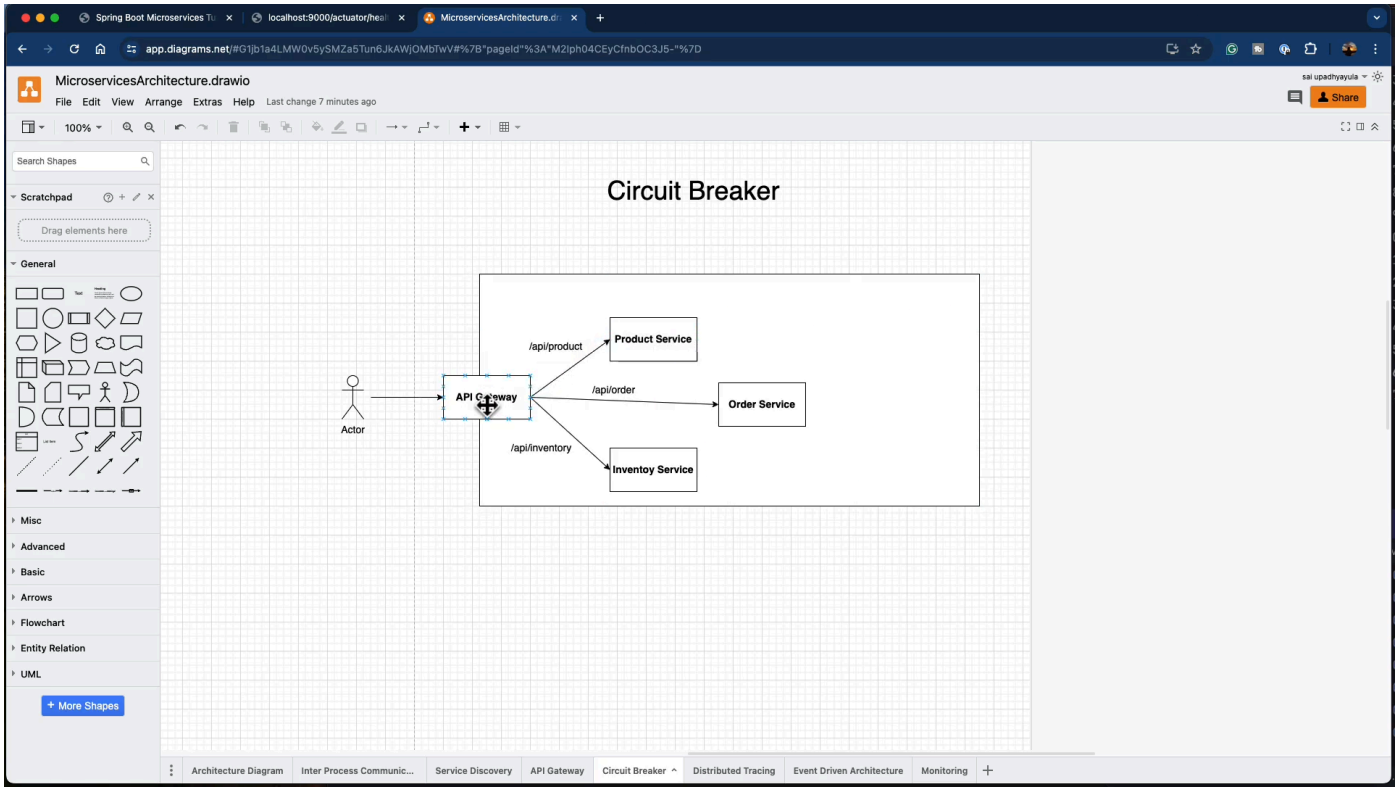
Overview

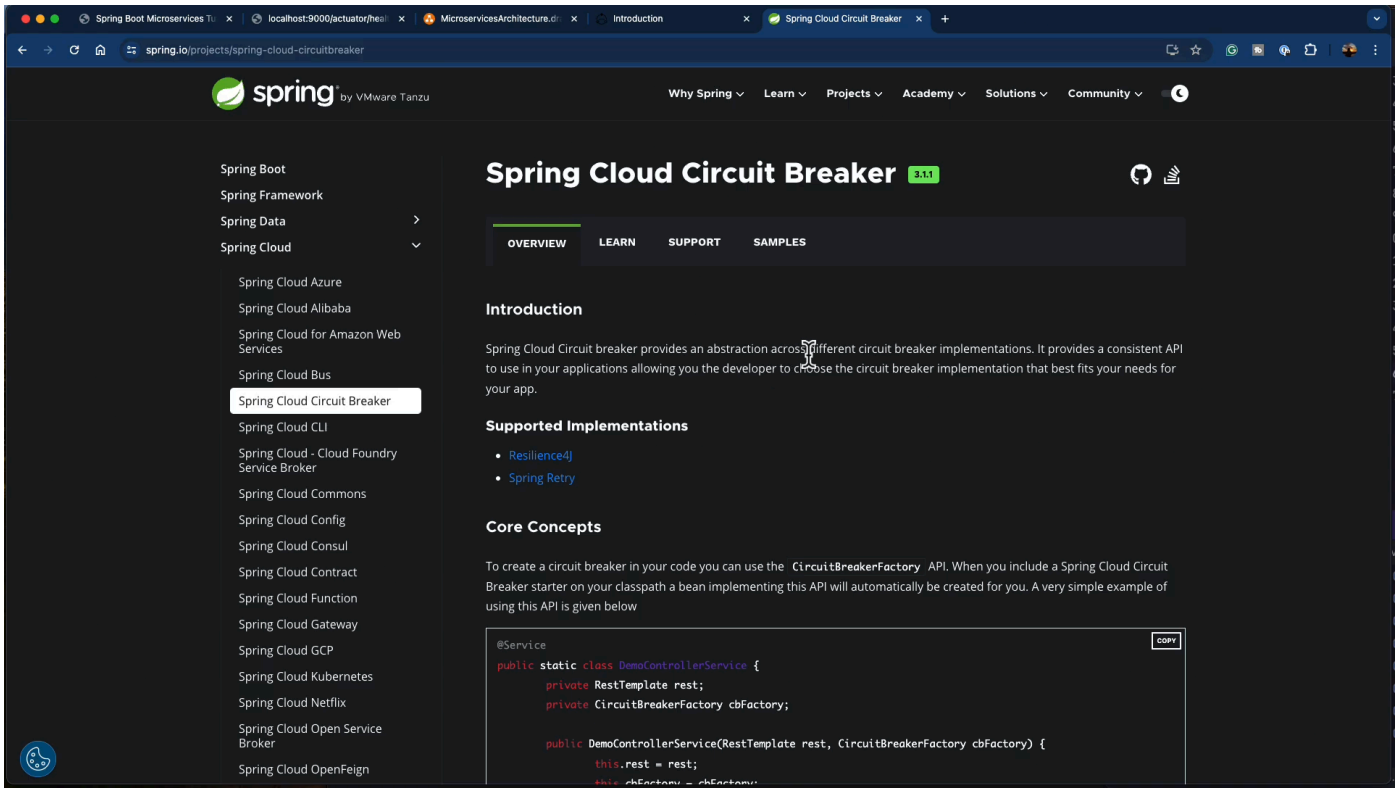
We will use **Resilience4j** to implement the following patterns:

- Rate Limiter
- Circuit Breaker
- Bulkhead Pattern

We will assume the use of Spring Cloud Circuit Breaker.


call / raise circuit open
success
fail [threshold reached]
Closed
Open
reset timeout
fail [under threshold]
fail
success
Half Open


MicroservicesArchitecture.drawio
Circuit Breaker
Actor
API Gateway
/api/product
Product Service
/api/order
Order Service
/api/inventory
Inventoy Service
Architecture Diagram
Inter Process Communic...
Service Discovery
API Gateway
Circuit Breaker
Distributed Tracing
Event Driven Architecture
Monitoring



1. Dependencies

Add these dependencies to the `pom.xml` of your service (e.g., Order Service):

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-reactor-
resilience4j</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. Configuration

Add the following configuration to your `application.properties`:

```
# Actuator Endpoints :
# Enables the circuit breaker health indicator to be shown in the health endpoint.
management.health.circuitbreakers.enabled=true
```

```
# Exposes all Actuator endpoints over HTTP (Web).
management.endpoints.web.exposure.include=*
# Shows full details of the health check (including nested components like db,
diskSpace, circuitBreakers).
management.endpoint.health.show-details=always

# Resilience4j Properties :

# Registers the circuit breaker health indicator with Spring Boot Actuator.
resilience4j.circuitbreaker.configs.default.register-health-indicator=true
# Sets the sliding window type to COUNT_BASED (records the outcome of the last N
calls).
resilience4j.circuitbreaker.configs.default.sliding-window-type=COUNT_BASED
# The size of the sliding window (number of calls to record).
resilience4j.circuitbreaker.configs.default.sliding-window-size=10
# The failure rate threshold in percentage. If 50% of calls fail, the circuit
opens.
resilience4j.circuitbreaker.configs.default.failure-rate-threshold=50
# The time to wait before transitioning from OPEN to HALF_OPEN state.
resilience4j.circuitbreaker.configs.default.wait-duration-in-open-state=5s
# The number of permitted calls when the circuit is in HALF_OPEN state.
resilience4j.circuitbreaker.configs.default.permitted-number-of-calls-in-half-open-
state=3
# Automatically transitions to HALF_OPEN state after the wait duration, without
waiting for a call.
resilience4j.circuitbreaker.configs.default.automatic-transition-from-open-to-half-
open-enabled=true
# Minimum calls before calculating failure rate
resilience4j.circuitbreaker.configs.default.minimum-number-of-calls=5

# Resilience4j Timeout Properties:
# The time to wait for a permission before timing out.
resilience4j.ratelimiter.configs.default.timeout-duration=3s
```

---

3. Deep Dive: Circuit Breaker Properties

Here's a detailed breakdown of each property, explaining logically what happens at runtime.

1️⃣ `register-health-indicator`

```
resilience4j.circuitbreaker.configs.default.register-health-indicator=true
```

What it does: Registers the **circuit breaker status** in **Spring Boot Actuator**. **Internal Logic:** The circuit breaker exposes its state (`CLOSED`, `OPEN`, `HALF_OPEN`), which

Actuator collects and displays at `/actuator/health`.

Example Output:

```
{
  "status": "DOWN",
  "components": {
    "circuitBreakers": {
      "status": "DOWN",
      "details": {
        "productService": {
          "state": "OPEN",
          "failureRate": "60%"
        }
      }
    }
  }
}
```

2️⃣ sliding-window-type

```
resilience4j.circuitbreaker.configs.default.sliding-window-type=COUNT_BASED
```

What it does: Defines **how failures are measured**.

- `COUNT_BASED`: Uses the outcome of the last **N calls**.
- `TIME_BASED`: Uses the outcome of calls in the last **T seconds**.

Example (COUNT_BASED): `Calls history: [✓ ❌ ❌ ✓ ✓ ❌ ✓ ✓ ❌ ❌]`
Only the count matters, not when they happened.

3️⃣ sliding-window-size

```
resilience4j.circuitbreaker.configs.default.sliding-window-size=10
```

What it does: Defines **N** = number of calls to observe. **Logic:** Every request outcome is recorded. When the size exceeds 10, the oldest call is removed.

Example: `Last 10 calls:` ✓ ✓ ❌ ❌ ❌ ✓ ✓ ❌ ✓ ❌ Failures = 5 / 10.

4 failure-rate-threshold

```
resilience4j.circuitbreaker.configs.default.failure-rate-threshold=50
```

What it does: Defines **when the circuit opens**. **Logic:** `failureRate = (failedCalls / totalCalls) × 100`. If `failureRate ≥ 50%`, the circuit transitions `CLOSED` → `OPEN`.

Example:

Total Calls	Failures	Failure Rate	Result
10	4	40%	CLOSED
10	5	50%	OPEN

5 wait-duration-in-open-state

```
resilience4j.circuitbreaker.configs.default.wait-duration-in-open-state=5s
```

What it does: Defines **how long the circuit stays OPEN**. **Logic:** When OPEN, all calls fail immediately. After 5s, the circuit becomes eligible for `HALF_OPEN`.

6 permitted-number-of-calls-in-half-open-state

```
resilience4j.circuitbreaker.configs.default.permitted-number-of-calls-in-half-open-state=3
```

What it does: Limits **how many test calls** are allowed during recovery (`HALF_OPEN` state). **Logic:** Ideally, allow a small number of calls to test if the backend is healthy. If they succeed, close the circuit. If they fail, re-open it.

7 minimum-number-of-calls

```
resilience4j.circuitbreaker.configs.default.minimum-number-of-calls=5
```

What it does: Defines the **minimum number of calls** that must be recorded **before the circuit breaker starts evaluating the failure rate**. **Logic:** Until this number is reached, the circuit stays `CLOSED` and failure rate is not calculated. This prevents premature opening on startup or low traffic volumes.

Timeline Example:

Call #	Result	Total Calls	Failure Rate	Circuit
1	❌	1	—	CLOSED
2	❌	2	—	CLOSED
3	❌	3	—	CLOSED
4	❌	4	—	CLOSED
5	❌	5	100%	🔴 OPEN

---

4. Rate Limiter & Time Limiter

Rate Limiter

```
resilience4j.ratelimiter.configs.default.timeout-duration=3s
```

What it does: How long a request waits for a **rate-limiter permission** (token). If the rate limit (e.g., 10 req/s) is exceeded, subsequent requests wait up to 3s for a permit before failing.

Time Limiter (`timeout-duration`)

```
resilience4j.timelimiter.configs.default.timeout-duration=3s
```

What it does: Limits **how long your service waits for a response from another service**. **Logic:** If the downstream service **does not respond within 3 seconds**, the call is canceled, marked as a failure, and triggers fallback logic.

Timeline:

1. Request sent.
2. Timer starts (3s).
3. If response < 3s → SUCCESS.
4. If response ≥ 3s → `TimeoutException` (Failure).

Relationship with Circuit Breaker: TimeLimiter failures count towards the Circuit Breaker's failure rate.

---

5. Retry Mechanism

Configuration

```
resilience4j.retry.configs.default.max-attempts=3
resilience4j.retry.configs.default.wait-duration=2s
```

`max-attempts`

Defines **how many total attempts** (original + retries) are made.

- **Example:** `max-attempts=3` means 1 original call + 2 retries.

`wait-duration`

Defines **how long to wait between attempts**.

- **Example:** Attempt 1 (fail) → wait 2s → Attempt 2 (fail) → wait 2s → Attempt 3.

Best Practices

- **Use Retry for:** Network glitches, temporary issues.
- **Avoid Retry for:** Overloaded services, long-running requests, deterministic errors (4xx).
- **Combine with Circuit Breaker:** Ideally, retry should wrap the circuit breaker or be wrapped by it depending on the desired behavior (see below).

---

6. Architectural Deep Dive: Filters & Gateway

In Spring Cloud Gateway (MVC), we use functional routing and filters.

Filter Chain Execution ("Russian Dolls" Model)

When you define filters:

```
.filter(Retry)
.filter(CircuitBreaker)
.filter(CaptureContext)
```

They wrap each other like nested functions:

```
Retry(
    CircuitBreaker(
        CaptureContext(
            Handler
        )
    )
)
```

Execution Order:

1. **Retry** starts.
2. **CircuitBreaker** starts (checks state).
3. **CaptureContext** (prepares error handling).
4. **Handler** executes HTTP call.

Failure Flow:

1. Handler throws exception.
2. CaptureContext catches, stores for fallback, rethrows.
3. CircuitBreaker records failure.
4. Retry catches, waits, and retries the whole chain (going back into CircuitBreaker).

`createServiceRoute(...)` Logic

The routing method typically does the following steps:

1. **Route Match:** Checks if path matches (e.g., `/api/product/**`).
2. **Service Resolution:** Uses Eureka/LoadBalancer to find the actual URL (e.g., `http://192.168.1.50:8080`).
3. **Rewrite & Forward:** Replaces the URL and forwards the request.
4. **Filters:** Applies the resiliency filters (Retry, Circuit Breaker).

Summary of Resilience Components

- **TimeLimiter** → "How long am I allowed to wait?"
- **RateLimiter** → "Can I send this request now?"
- **CircuitBreaker** → "Should I even try?"
- **LoadBalancer** → "Where do I send it?"
- **Retry** → "Maybe it will work if I try again?"

7. Deep Dive: Handling HTTP Failures (The 500/404 Problem)

A common challenge with Circuit Breakers in API Gateways is that **HTTP 5xx (Server Error)** and **404 (Not Found)** responses are technically "successful" network calls. They do not throw generic Java Exceptions (like `ConnectException`), so **Resilience4j does not count them as failures by default**.

To fix this, we implement a **Translation Pattern**: converting specific HTTP Status Codes into Java Exceptions *before* the Circuit Breaker sees the response.

1. The Strategy: Inner Translation Filter

We introduce a dedicated filter, `ExceptionTranslationFilter`, which sits **inside** the Circuit Breaker scope.

Execution Chain (The "Russian Doll" Model): When filters are defined in `Routes.java`, they wrap each other. The **last** defined filter is the **outermost** wrapper.

1. **Retry Filter (Outer)**: Wraps everything. Catches exceptions from the inner chain to trigger retries.
2. **Circuit Breaker (Middle)**: Wraps the inner logic. Monitors for exceptions to record failures/successes.
3. **ExceptionTranslationFilter (Inner)**: Wraps the actual handler. Inspects the HTTP response. If it's a 5xx/404, it **throws an exception** so the upper layers (CB and Retry) can react.
4. **Service (Handler)**: The actual network call.

2. Implementation: `ExceptionTranslationFilter`

This filter inspects the response status and throws a `CustomAppException`.

```
public class ExceptionTranslationFilter {
    private static final Logger log =
LoggerFactory.getLogger(ExceptionTranslationFilter.class);

    public static HandlerFilterFunction<ServerResponse, ServerResponse>
checkStatus() {
        return (request, next) -> {
            ServerResponse response = next.handle(request);

            // 1. Map 5xx Server Errors to CustomAppException
            if (response.statusCode().is5xxServerError()) {
                log.error("Upstream service returned server error: {}",
response.statusCode());
                throw new CustomAppException(
                        HttpStatus.resolve(response.statusCode().value()),
                        "Upstream service returned server error: " +
response.statusCode()
                );
            }

            // 2. Map 404 Not Found to CustomAppException (Optional)
            if (response.statusCode() == HttpStatus.NOT_FOUND) {
                log.warn("Upstream service returned Not Found (404)");
```

```
                throw new CustomAppException(
                        HttpStatus.NOT_FOUND,
                        "Upstream service returned Not Found (404)"
                );
            }
            return response;
        };
    }
}
```

3. Implementation: Custom Exception

```
public class CustomAppException extends RuntimeException {
    private final HttpStatus status;
    // ... constructors ...
}
```

4. Correct Filter Ordering (`Routes.java`)

To ensure the Circuit Breaker records the failure, the definition order in `Routes.java` is critical.

```
private RouterFunction<ServerResponse> createServiceRoute(String serviceName,
String pathPrefix) {
    return GatewayRouterFunctions.route(serviceName)
            .route(RequestPredicates.path(pathPrefix), request -> { ... })

            // 1. INNERMOST: Checks Status -> Throws Exception
            .filter(ExceptionTranslationFilter.checkStatus())

            // 2. MIDDLE: Circuit Breaker -> Catches Exception -> Records Failure -
> Rethrows
            .filter(CircuitBreakerFilterFunctions.circuitBreaker(
                    serviceName + "CircuitBreaker",
                    URI.create("forward:/fallbackRoute/" + serviceName)))

            // 3. OUTERMOST: Retry -> Catches Exception -> Retries
            .filter(Resilience4jRetryFilter.retry("default", retryRegistry))

            .build();
}
```

---

8. Comprehensive Execution Flow Analysis (Corrected)

The "Inner Retry" Logic

In `Routes.java`, filters are applied in the order they are defined effectively creating a chain where the **first** defined filter wraps the **second**, and so on.

Current Filter Chain:

1. `ExceptionTranslationFilter` (Wraps everything)
2. `CircuitBreaker` (Wraps Retry)
3. `Retry` (Innermost - Wraps Handler)

Implication:

- **Circuit Breaker** calls **Retry**.
- **Retry** calls **Handler** (and loops if needed).
- **Result:** The Circuit Breaker only sees the **final outcome** of the Retry loop.
    - If Retry succeeds on attempt #3 -> Circuit Breaker sees **SUCCESS**.
    - If Retry fails all 3 attempts -> Circuit Breaker sees **FAILURE (1 Count)**.

Scenario: Upstream Service Returns HTTP 500 (Internal Server Error)

Step 1: The Request Begins

- **ExceptionTranslationFilter** calls `CircuitBreaker`.
- **Circuit Breaker** (State: CLOSED) calls `Retry`.
- **Retry** calls **Handler** (Attempt #1).

Step 2: Failure & Retry Loop (Hidden from CB)

- **Handler** returns 500.
- **Retry Filter** logic detects 500 (via internal check or `ExceptionTranslation` if we placed it inside, but here Retry logic handles the loop).
    - **Action:** Retry decides to retry. Sleeps 2s.

    - **Circuit Breaker is unaware** of this specific failure yet; it's still waiting for `Retry` to return.
- **Retry** calls **Handler** (Attempt #2). Fails (500). Sleeps 2s.
- **Retry** calls **Handler** (Attempt #3). Fails (500).

Step 3: Retry Exhaustion

- **Retry** gives up. Throws `CustomAppException` (or bubbles the last 500 up).

Step 4: Circuit Breaker Recording

- **Circuit Breaker** finally receives the Exception (after 6 seconds of retrying).
- **Action:** Records **1 FAILURE**.
- **State Update:** `failedCalls` increments by 1.

Step 5: Translation & Fallback

- The exception bubbles up to `ExceptionTranslationFilter` (which might log it).
- Ultimately traps to the Fallback URI defined in Circuit Breaker configuration (e.g., `forward:/fallbackRoute/...`).

---

9. Configuration Details & Design Choices

Retry vs Circuit Breaker Order

The order determines how failures are counted.

Order	Structure	Behavior	Use Case
Retry(CB) *(Retry Wraps CB)*	`Retry` calls `CB`	If CB fails, Retry calls it again. **3 Retries = 3 CB Failures.**	Aggressive. Opens Circuit Breaker very fast.
CB(Retry) *(Current)*	`CB` calls `Retry`	CB waits for Retry loop.	Standard. "Try hard before giving up".

Order	Structure	Behavior	Use Case
		3 Retries = 1 CB Failure.	

Current Configuration (`application.properties`)

```
resilience4j.retry.configs.default.max-attempts=3
resilience4j.retry.configs.default.wait-duration=2s
```

- **Logic**: 1 Initial + 2 Retries.
- **CB Impact**: Takes ~4s+ to fail 1 request. Circuit Breaker only counts this as **one** failed call.

Note on TimeLimiter: Since `CB` wraps `Retry`, the `TimeLimiter` (if associated with CB) applies to the **entire retry loop**.

- If `TimeLimiter=3s` and `Retry=4s` (2s x 2), the **TimeLimiter will cut the retries short**.
- You must ensure `TimeLimiter > (max-attempts * wait-duration) + execution-time`.

---

10. Failure Scenarios Matrix

Scenario	Trigger Mechanism	Who Detects It?	Flow Summary
1. Timeout	**TimeLimiter**	**Gateway Level**	1. CB starts timer. 2. Retry loop takes too long. 3. **TimeLimiter kills** the request. 4. CB records Failure (Timeout).
2. No Server	**Netty/Io Error**	**Network Level**	1. Retry loops on Connection Refused. 2. Retry exhausts.

Scenario	Trigger Mechanism	Who Detects It?	Flow Summary
			3. Throws `ConnectException`. 4. CB records 1 Failure.
3. Service 500	**Status Logic**	**App Level**	1. Retry loops on 500. 2. Retry exhausts. 3. Throws `CustomAppException`. 4. CB records 1 Failure.